

# ZYProject 网络请求优化建议

## ZYProject 网络请求实现的优化建议

### 一、重构重复代码

当前代码中，`post:` 与 `postNoPrompt:`、`postBean:` 与 `postBeanNoPrompt:` 等方法存在大量重复逻辑（如响应处理、提示显示）。建议提取公共方法，减少冗余，提升可维护性。

### 优化方案：提取公共响应处理方法

```
// 1. 公共响应处理方法：统一处理服务器响应与提示逻辑
+ (void)handleResponse:(id)response showSuccessPrompt:(BOOL)showPrompt completion:(
void(^)(ZYNetWorkModel *model))completion {
    // 1.1 JSON数据转换为模型（使用YYModel）
    ZYNetWorkModel *model = [ZYNetWorkModel yy_modelWithJSON:response];

    // 1.2 根据状态码处理不同场景
    if ([model.status isEqualToString:@"1"]) {
        // 状态1：请求成功（按需显示成功提示）
        if (showPrompt) {
            [EasyTextView showSuccessText:@"成功!"];
        }
    } else if ([model.status isEqualToString:@"2"]) {
        // 状态2：业务错误（显示错误提示）
        [EasyTextView showErrorText:model.message];
    } else if ([model.status isEqualToString:@"3"]) {
        // 状态3：登录失效（统一跳转登录页，避免重复代码）
        [self jumpToLoginPage];
    }

    // 1.3 回调返回模型数据
    if (completion) {
        completion(model);
    }
}
```

```

}
// 辅助方法：登录失效时跳转登录页
+ (void)jumpToLoginPage {
    UIViewController *loginVC = [[LoginViewController alloc] init];
    // 获取当前顶层控制器，避免导航层级问题
    UIViewController *topVC = [UIApplication sharedApplication].keyWindow.
rootViewController;
    while (topVC.presentedViewController) {
        topVC = topVC.presentedViewController;
    }
    [topVC presentViewController:loginVC animated:YES completion:nil];
}
// 2. 简化POST请求方法（复用公共逻辑）
+ (void)post:(BADataEntity *)baDataEntity success:(void (^)(ZYNetWorkModel *))success
failure:(void (^)(NSError *))failure {
    [self requestWithEntity:baDataEntity
        method:@"POST"
        showSuccessPrompt:YES // 显示成功提示
        success:success
        failure:failure];
}
+ (void)postNoPrompt:(BADataEntity *)baDataEntity success:(void (^)(ZYNetWorkModel *))
success failure:(void (^)(NSError *))failure {
    [self requestWithEntity:baDataEntity
        method:@"POST"
        showSuccessPrompt:NO // 不显示成功提示
        success:success
        failure:failure];
}
}

```

## 二、统一网络请求底层实现

当前项目混用 `BANetManager`、`NSURLConnection`、`AFHTTPSessionManager` 三种网络框架，导致维护成本高、逻辑分散。建议统一使用 **AFHTTPSessionManager**（iOS 主流网络库，支持请求取消、进度监听、HTTPS 配置，稳定性与扩展性更强）。

### 优化方案：创建统一的 AFHTTPSessionManager 实例

```

// 1. 单例模式：创建全局唯一的网络请求管理器
+ (AFHTTPSessionManager *)sharedSessionManager {
    static AFHTTPSessionManager *manager = nil;
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        // 1.1 初始化管理器（默认基础URL，可根据需求配置）
        manager = [AFHTTPSessionManager manager];

        // 1.2 基础配置：超时时间、序列化方式
        manager.requestSerializer.timeoutInterval = 15; // 统一超时时间15秒
        manager.requestSerializer = [AFJSONRequestSerializer serializer]; // 请求体JSON格式
        manager.responseSerializer = [AFJSONResponseSerializer serializer]; // 响应JSON格式

        // 1.3 设置公共请求头（如Content-Type、Accept）
        [manager.requestSerializer setValue:@"application/json" forHTTPHeaderField:@"Content-Type"];
        [manager.requestSerializer setValue:@"application/json" forHTTPHeaderField:@"Accept"];
    }];

    // 1.4 添加公共参数（如Token，从本地存储获取）
    NSString *token = [[NSUserDefaults standardUserDefaults] stringForKey:@"userToken"];
    if (token) {
        [manager.requestSerializer setValue:[NSString stringWithFormat:@"Bearer %@", token]
        forHTTPHeaderField:@"Authorization"];
    }

    // 1.5 HTTPS配置（信任所有证书，适合测试环境；生产环境需配置SSL证书）
    AFSecurityPolicy *securityPolicy = [AFSecurityPolicy policyWithPinningMode:
    AFSSLPinningModeNone];
    securityPolicy.allowInvalidCertificates = YES;
    securityPolicy.validatesDomainName = NO;
    manager.securityPolicy = securityPolicy;
    });
    return manager;
}

// 2. 统一请求方法：支持POST/GET，复用底层逻辑
+ (void)requestWithEntity:(BADATAEntity *)entity
    method:(NSString *)method
    showSuccessPrompt:(BOOL)showPrompt
    success:(void (^)(ZYNetWorkModel *))success
    failure:(void (^)(NSError *))failure {

```

```

// 显示加载状态
[SVProgressHUD show];

// 获取单例管理器
AFHTTPSessionManager *manager = [self sharedSessionManager];

// 发起网络请求（根据method区分POST/GET）
NSURLSessionDataTask *task = [manager dataTaskWithHTTPMethod:method
                                urlString:entity.urlString
                                parameters:entity.parameters
                                headers:nil
                                uploadProgress:nil
                                downloadProgress:nil
                                success:^(NSURLSessionDataTask * _Nonnull task, id _Nullable
responseObject) {
    // 主线程处理UI（隐藏加载框、回调结果）
    dispatch_async(dispatch_get_main_queue(), ^{
        [SVProgressHUD dismiss];
        // 调用公共响应处理方法
        [self handleResponse:responseObject showSuccessPrompt:
showPrompt completion:success];
    });
}
failure:^(NSURLSessionDataTask * _Nullable task, NSError * _
Nonnull error) {
    // 主线程处理UI（隐藏加载框、显示错误）
    dispatch_async(dispatch_get_main_queue(), ^{
        [SVProgressHUD dismiss];
        // 错误提示（调用统一错误处理方法）
        [EasyTextView showErrorText:[self isNetWorking:error]];
        // 回调错误信息
        if (failure) {
            failure(error);
        }
    });
}];

// 启动任务
[task resume];
}

```

## 三、添加请求取消机制

当前实现缺少“请求取消”功能，在列表刷新、页面销毁时，未完成的请求会继续占用资源，可能导致数据错乱或内存泄漏。建议添加“请求管理”功能，支持按标识符取消单个请求或所有请求。

### 优化方案：请求管理与取消

```
// 1. 扩展ZYNetworkRequest类，添加任务管理字典
@interface ZYNetworkRequest ()
// 存储任务：key=任务标识符，value=网络任务
@property (nonatomic, strong) NSMutableDictionary<NSString *, NSURLSessionDataTask *> *
tasks;
@end
@implementation ZYNetworkRequest
// 2. 单例初始化时初始化任务字典
+ (instancetype)sharedInstance {
    static ZYNetworkRequest *instance = nil;
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        instance = [[ZYNetworkRequest alloc] init];
        instance.tasks = [NSMutableDictionary dictionary]; // 初始化任务字典
    });
    return instance;
}
// 3. 修改请求方法，支持传入任务标识符
+ (NSURLSessionDataTask *)requestWithEntity:(BADDataEntity *)entity
    method:(NSString *)method
    taskIdentifier:(NSString *)identifier // 新增：任务标识符（如“userList_1”）
    showSuccessPrompt:(BOOL)showPrompt
    success:(void (^)(ZYNetWorkModel *))success
    failure:(void (^)(NSError *))failure {
    // 3.1 取消相同标识符的旧任务（避免重复请求）
    if (identifier) {
        [self cancelRequestWithIdentifier:identifier];
    }

    // 3.2 发起请求（复用统一请求方法）
    AFHTTPSessionManager *manager = [self sharedSessionManager];
```

```

NSURLSessionDataTask *task = [manager dataTaskWithHTTPMethod:method
                                urlString:entity.urlString
                                parameters:entity.parameters
                                headers:nil
                                uploadProgress:nil
                                downloadProgress:nil
                                success:^(NSURLSessionDataTask * _Nonnull task, id _Nullable
responseObject) {
    // 成功后移除任务（避免内存泄漏）
    if (identifier) {
        [[self sharedInstance].tasks removeObjectForKey:identifier];
    }
    // 后续处理...
}
failure:^(NSURLSessionDataTask * _Nullable task, NSError * _
Nonnull error) {
    // 失败后移除任务
    if (identifier) {
        [[self sharedInstance].tasks removeObjectForKey:identifier];
    }
    // 后续处理...
}];

// 3.3 保存任务到字典（按标识符关联）
if (identifier) {
    [[self sharedInstance].tasks setObject:task forKey:identifier];
}

[task resume];
return task;
}

// 4. 取消指定标识符的请求
+ (void)cancelRequestWithIdentifier:(NSString *)identifier {
    if (!identifier) return;

    // 从字典中获取任务并取消
    NSURLSessionDataTask *task = [[self sharedInstance].tasks objectForKey:identifier];
    if (task && task.state != NSURLSessionTaskStateCanceling && task.state !=
NSURLSessionTaskStateCompleted) {
        [task cancel]; // 取消任务
        [[self sharedInstance].tasks removeObjectForKey:identifier]; // 移除任务
    }
}

```

```

    }
}
// 5. 取消所有未完成的请求（如页面销毁时调用）
+ (void)cancelAllRequests {
    for (NSURLSessionDataTask *task in [[self sharedInstance].tasks allValues]) {
        if (task.state != NSURLSessionTaskStateCanceling && task.state != NSURLSessionTaskStateCompleted) {
            [task cancel];
        }
    }
    [[self sharedInstance].tasks removeAllObjects]; // 清空任务字典
}
@end
// 使用示例：列表页面请求取消
- (void)dealloc {
    // 页面销毁时，取消“用户列表”相关请求
    [ZYNetworkRequest cancelRequestWithIdentifier:@"userListRequest"];
}

```

## 四、优化线程管理

当前代码中，部分 UI 操作（如 `SVProgressHUD` 显示 / 隐藏、`EasyTextView` 提示）可能在异步线程执行，导致界面卡顿、崩溃或提示不显示。需确保 **所有 UI 操作都在主线程执行**（iOS 规定 UI 操作必须在主线程）。

### 优化方案：强制主线程执行 UI 操作

```

// 错误示例：可能在异步线程执行UI操作
[SVProgressHUD show]; // 若在网络回调（异步线程）中调用，可能导致问题
// 正确示例1：直接在主线程执行（如请求发起前）
[SVProgressHUD show]; // 请求发起前在主线程调用，安全
// 正确示例2：网络回调中（异步线程），通过dispatch_async切换到主线程
success:^(NSURLSessionDataTask * _Nonnull task, id _Nullable responseObject) {
    dispatch_async(dispatch_get_main_queue(), ^{
        // 所有UI操作放在主线程
        [SVProgressHUD dismiss]; // 隐藏加载框
        [EasyTextView showSuccessText:@"成功!"]; // 显示成功提示
        if (success) {

```

```

        success(model); // 回调结果（若回调中包含UI操作，也需确保在主线程）
    }
    });
}
// 正确示例3：工具类方法中，确保UI操作在主线程
+ (void)showSuccessToast:(NSString *)message {
    dispatch_async(dispatch_get_main_queue(), ^{
        [EasyTextView showSuccessText:message];
    });
}

```

## 五、添加参数验证和安全检查

当前实现缺少“请求参数有效性验证”，若传入空 URL、空参数，会导致无效请求，浪费资源且难以调试。建议添加参数验证，提前拦截错误，提升稳定性。

### 优化方案：参数验证与错误拦截

```

// 1. 公共参数验证方法
+ (BOOL)validateRequestParams:(BADDataEntity *)entity error:(NSError **)error {
    // 1.1 验证entity非空
    if (!entity) {
        if (error) {
            *error = [NSError errorWithDomain:@"ZYNetworkError"
                                   code:-1
                                   userInfo:@{NSLocalizedStringKey:@"请求参数实体为空"}];
        }
        return NO;
    }

    // 1.2 验证URL非空且格式正确
    if (!entity.urlString || entity.urlString.length == 0) {
        if (error) {
            *error = [NSError errorWithDomain:@"ZYNetworkError"
                                   code:-2
                                   userInfo:@{NSLocalizedStringKey:@"请求URL为空"}];
        }
        return NO;
    }
}

```

```

}
// 验证URL格式（简单正则匹配）
NSString *urlRegex = @"^https?:/*.*$";
NSPredicate *predicate = [NSPredicate predicateWithFormat:@"SELF MATCHES %@",
urlRegex];
if (![predicate evaluateWithObject:entity.urlString]) {
    if (error) {
        *error = [NSError errorWithDomain:@"ZYNetworkError"
                                code:-3
                                userInfo:@{NSLocalizedDescriptionKey:@"请求URL格式错误（需http/https开
头）"}];
    }
    return NO;
}

// 1.3 验证必要参数（根据业务需求，如POST请求参数非空）
if ([entity.method isEqualToString:@"POST"] && (!entity.parameters || entity.parameters.
count == 0)) {
    if (error) {
        *error = [NSError errorWithDomain:@"ZYNetworkError"
                                code:-4
                                userInfo:@{NSLocalizedDescriptionKey:@"POST请求参数为空"}];
    }
    return NO;
}

// 验证通过
return YES;
}

// 2. 在请求方法中调用参数验证
+ (void)post:(BADataEntity *)baDataEntity success:(void (^)(ZYNetWorkModel *))success
failure:(void (^)(NSError *))failure {
    // 2.1 参数验证
    NSError *validationError = nil;
    if (![self validateRequestParams:baDataEntity error:&validationError]) {
        // 验证失败：显示错误提示并回调
        [EasyTextView showErrorText:validationError.localizedDescription];
        if (failure) {
            failure(validationError);
        }
    }
    return; // 拦截无效请求，不发起网络调用
}

```

```

}

// 2.2 验证通过，发起请求
[self requestWithEntity:baDataEntity
    method:@"POST"
    showSuccessPrompt:YES
    success:success
    failure:failure];
}

```

## 六、添加网络缓存机制

对于高频且不实时的数据（如商品列表、分类信息），建议添加本地缓存，减少重复网络请求，提升加载速度与离线体验。利用 AFNetworking 结合 `NSCache` 实现缓存功能。

### 优化方案：GET 请求缓存支持

```

// 1. 扩展ZYNetworkRequest，添加缓存相关属性与方法
@interface ZYNetworkRequest ()
@property (nonatomic, strong) NSCache *responseCache; // 内存缓存
@property (nonatomic, strong) NSString *cachePath; // 磁盘缓存路径
@end

@implementation ZYNetworkRequest
// 2. 单例初始化时配置缓存
+ (instancetype)sharedInstance {
    static ZYNetworkRequest *instance = nil;
    static dispatch_once_t onceToken;
    dispatch_once(&onceToken, ^{
        instance = [[ZYNetworkRequest alloc] init];
        // 2.1 初始化内存缓存（设置最大缓存大小10MB）
        instance.responseCache = [[NSCache alloc] init];
        instance.responseCache.totalCostLimit = 10 * 1024 * 1024;

        // 2.2 初始化磁盘缓存路径（沙盒Library/Caches/NetworkCache）
        NSString *cacheDir = [NSSearchPathForDirectoriesInDomains(NSCachesDirectory,
            NSUserDomainMask, YES) firstObject];
        instance.cachePath = [cacheDir stringByAppendingPathComponent:@"NetworkCache"];
        // 创建缓存目录（若不存在）
    });
}

```

```
NSFileManager *fileManager = [NSFileManager defaultManager];  
if (![fileManager fileExistsAtPath:instance.cachePath]) {  
    [fileManager createDirectoryAtPath:instance.cachePath withIntermediateDirectories:YES  
attributes:nil error:nil];  
}
```

（注：文档部分内容可能由 AI 生成）